ARCHITECTURE & IMPLEMENTATION PLAN | FLUTTER LINUX DESKTOP | HIGH PERFORMANCE CA

Cellular Automata Desktop Application (CA Studio)

Decoupled Isolate Simulation Engine, GPU Fragment Shader Viewport, and Embedded RuleEditor Dialog Integration

Document Version: 1.0

Target Scale: 1200 × 800 (960,000 cells)

Target Frame Rate: 60 – 144 FPS

Rule Editor Subfolder: `RuleEditor/`

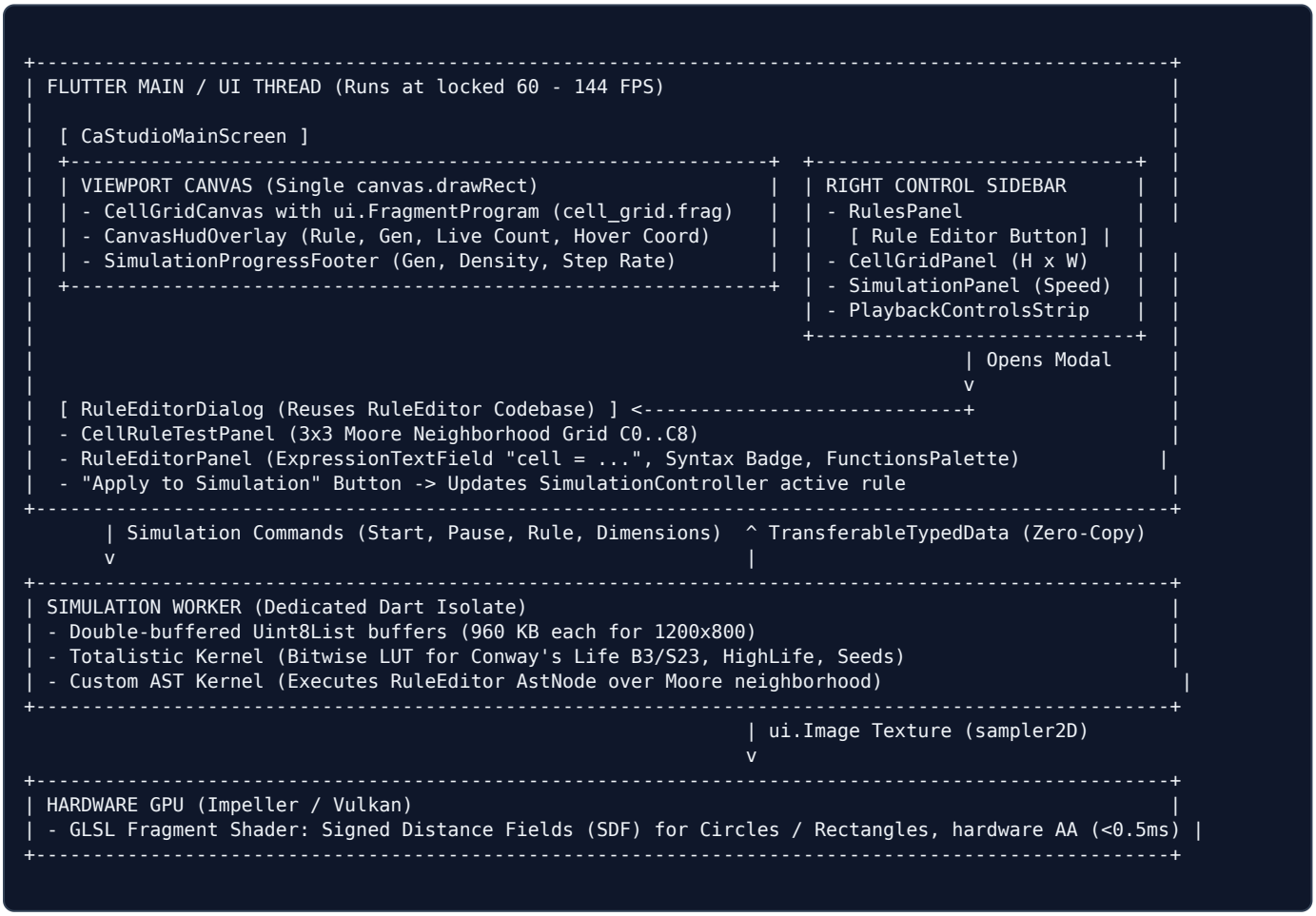
1. Executive Summary & Core Objectives

The **CA Studio** application is a professional workstation tool built with Flutter for Linux Desktop designed to simulate large-scale 2D cellular automata. It resolves the computational and rendering bottlenecks of high-density grids while delivering an intuitive desktop interface faithful to the UX specifications.

> **The Three Architectural Pillars**
>
> 1. **Decoupled Compute (Background Dart Isolate):** Evaluates cellular transitions across up to 960,000 cells every 10–100 ms using zero-allocation double buffers.
> 2. **Zero-Jank GPU Fragment Shader (Impeller / Vulkan):** Translates the raw state buffer into a 2D texture (`sampler2D`) and executes **exactly one draw call** with signed distance fields (SDF) for silky 60–144 FPS pan, zoom, and interaction.
> 3. **Embedded Rule Editor Modal Dialog:** Reuses the full grammar lexer, recursive descent parser, AST evaluator, functions palette, and 3x3 Moore neighborhood matrix from the `RuleEditor` codebase.

2. System Architecture Blueprint

The application strictly separates compute cycles from the rendering and interaction clock. The UI thread never performs cell grid loops, and the compute isolate never touches UI objects.



3. Right Sidebar & Rule Editor Dialog Interaction Flow

The right sidebar faithfully fulfills the hand-drawn sketch and Stitch UX layout. In the **Rules** panel, the primary button launches the Rule Editor dialog:

1. Rules Sidebar Panel

Preset Dropdown: Conway's Life (B3/S23), HighLife, Seeds, Brian's Brain, Day & Night, Custom.

Parameter Chips: `SURVIVE: 2, 3`, `BIRTH: 3`, `STATES: 2`.

Action: Tapping `[ Rule Editor (Configure ->)]` opens the modal workstation dialog without leaving the app.

2. Embedded RuleEditorDialog

Workstation Modal: 1100 × 700 dark workstation overlay.

Left Panel: Interactive 3x3 Moore grid ($C_0 \dots C_8$), toggleable states, and `Apply Rule` button.

Right Panel: Monospaced expression editor with `cell =` prefix, syntax badge, and functions palette.

Apply Action: Validates AST, updates simulation, and closes.

4. Isolate Communication & Zero-Copy Protocol

Dart isolates maintain isolated heaps. To avoid deep-copy memory latency of 960,000 bytes every 10–30 ms, the system utilizes `TransferableTypedData` for $O(1)$ pointer transfer.

Phase	Message / Channel	Description & Responsibilities
1. Handshake	`ReceivePort` → `SendPort`	UI thread spawns isolate with its `SendPort`. Isolate replies with its own command `SendPort`.
2. UI Commands	`StartSim`, `PauseSim`, `SetRule`, `Resize`, `Randomize`	UI dispatches lightweight control objects to isolate. Simulation tick begins or updates immediately.
3. Frame Emission	`SimulationFrameMessage` via `TransferableTypedData`	Isolate wraps `Uint8List` into `TransferableTypedData.fromList([nextBuffer])` and sends to UI port in $O(1)$ time.
4. UI Ingestion	`materialize().asUint8List()` → `ui.Image`	UI thread converts bytes into hardware texture via `ui.ImmutableBuffer` and `ImageDescriptor.raw`.
5. Backpressure	Drop-frame policy	If UI is busy during window resize, intermediate frames drop cleanly. The UI always paints the freshest state.

5. Key Classes & Modular Responsibilities

Class Name	Layer / Location	Core Responsibilities
`CaStudioApp`	Presentation (Main)	Desktop window configuration, minimum window bounds (1200 × 800), global theme setup.
`CaStudioMainScreen`	Presentation (Screen)	Main workstation layout coordinating header, canvas viewport, timeline footer, and sidebar.
`StudioWindowHeader`	Presentation (Header)	Window chrome, file breadcrumb, zoom controls (`- 100% +` , `Center`), and real-time FPS badge.
`CellGridViewport`	Presentation (Viewport)	Handles canvas pan, zoom, interactive mouse draw/erase, and coordinate translation.
`CellGridCanvas`	Presentation (CustomPainter)	Single `canvas.drawRect` draw call binding `ui.FragmentProgram` and `u_grid_tex` texture.
`CanvasHudOverlay`	Presentation (HUD)	Top-left viewport HUD: active rule name, generation count, live cell count, and hovered `(X, Y)` coordinate.
`SimulationProgressFooter`	Presentation (Timeline)	Timeline progress bar matching wireframe sketch: `Gen 428 / 1,000` , density %, and step ms.
`ControlSidebar`	Presentation (Sidebar)	Houses `RulesPanel` , `CellGridPanel` , `SimulationPanel` , and `PlaybackControlsStrip` .
`RuleEditorDialog`	Presentation (Modal Dialog)	Workstation modal embedding `CellRuleTestPanel` and `RuleEditorPanel` from `RuleEditor/` .
`SimulationController`	State Management	Coordinates simulation state (play, pause, step), grid geometry, rule AST, and telemetry.
`RuleStudioController`	State (Reused from RuleEditor)	Drives AST parsing, syntax validation, token insertion, and 3x3 Moore neighborhood testing.
`SimulationIsolateWorker`	Compute Engine (Isolate)	Background thread executing cellular transitions over double-buffered `Uint8List` arrays.
`TotalisticKernel`	Compute Kernel	Fast-path bitwise lookup table for standard Life-like rules (Conway B3/S23, HighLife, etc.).
`AstEvaluatorKernel`	Compute Kernel	Custom rule engine evaluating the `AstNode` expression tree over neighborhood states.
`cell_grid.frag`	GPU Shader (GLSL)	Hardware fragment shader computing SDF anti-aliased circles or rectangles with zero-cost palette mapping.

6. Implementation Roadmap & Milestones

Phase 1: Project Setup & Code Sharing

• Initialize Flutter desktop application inside `CellAutomata/`.

• Add `RuleEditor` as a local path dependency (`path: ../RuleEditor`) in `pubspec.yaml`.

• Compile and configure the GLSL fragment shader asset (`cell_grid.frag`).

Phase 2: Isolate Engine & Zero-Copy IPC

• Implement `SimulationIsolateWorker` with double-buffered `Uint8List` arrays.

• Build `TotalisticKernel` and `AstEvaluatorKernel`.

• Wire two-way `SendPort` / `ReceivePort` with `TransferableTypedData`.

Phase 3: GPU Shader Viewport & Canvas

• Implement `CellGridCanvas` with single draw call `canvas.drawRect`.

• Bind `u_grid_tex` texture created from isolate buffer.

• Implement pan, zoom, coordinate mapping, and HUD overlay.

Phase 4: Sidebar Controls & Timeline

• Build right sidebar: Rules, Grid Size, Simulation parameters, and Playback strip.

• Implement bottom shaded timeline progress bar with real-time metrics.

• Wire playback lifecycle (Play, Pause, Step, Rewind, Multipliers).

Phase 5: Rule Editor Dialog Integration & Verification

• Construct `RuleEditorDialog` embedding `CellRuleTestPanel` and `RuleEditorPanel` from `RuleEditor/`.

• Wire the `[ Rule Editor]` button to open the modal dialog.

• Implement "Apply to Simulation" action to pass the compiled `CellRule` / `AstNode` directly to the isolate worker.

• Verify 60+ FPS performance on 1200×800 grid and seamless rule execution.